

Cascade Replication 셋업 매뉴얼 — Hybrid DR Project

처음 셋업하는 사람을 위한 step-by-step 가이드. 우리 팀이 실제로 부딪힌 에러 9개와 해결법 포함. 환경: 온프레임 VMware (HAProxy/WEBWAS/DB) + AWS (Bastion/Jenkins/HAProxy/DB EC2/RDS) + Tailscale subnet router. 작성: 2026-04-26 cascade replication 디버깅 결과 정리.

0. 시작하기 전 — 이 매뉴얼이 가정하는 것

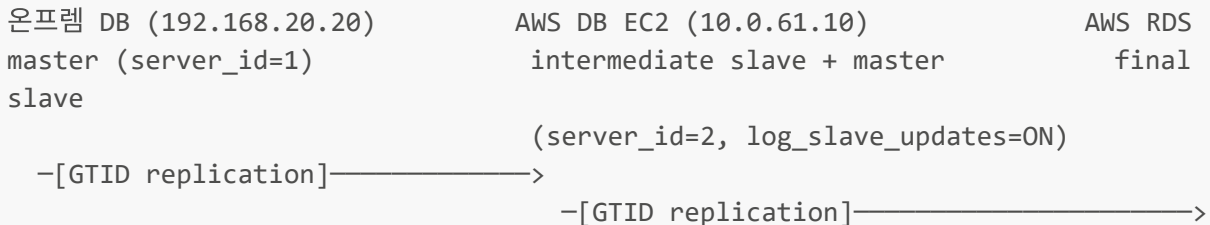
인프라 (이미 셋업돼있어야 함)

- ✓ AWS EC2 4대 (Bastion / Jenkins / HAProxy / DB EC2) + RDS 가동 중
- ✓ 온프레임 VMware 3대 (HAProxy / WEBWAS / DB) 가동 중
- ✓ HAProxy(온프레임) 가 Tailscale subnet router 로 192.168.20.0/24 advertise
- ✓ AWS DB EC2 가 Tailscale 클라이언트로 위 subnet 라우팅 받음
- ✓ Terraform/Ansible 코드 (commit 440faa7 이후) 가 push 됨

코드 / 레포지토리

- ✓ App 레포 (Soldesk-Cloud/App) 의 main 브랜치에 `release/app.jar` + `sql/schema.sql` 있음
- ✓ App 레포 `application.yml` 의 `datasource` 가 `inventory DB` / `app_user` / `Soldesk1`. 박혀있음
- ✓ Ansible/Jenkinsfile/user_data 의 인프라 변수가 위 값들과 align

셋업 후 동작 목표



라이브 검증 시 온프레임 INSERT → 5초 안에 RDS 에 도달.

1. 전체 흐름 도식

[Phase A] 온프레임 DB 준비

- `my.cnf` 의 `!includedir` 확인 + GTID 활성화
- `inventory DB` + `app_user` + `repl_user` 생성
- App 레포의 `sql/schema.sql` 적용

[Phase B] AWS DB EC2 cascade slave 셋업

- mysql 셋업은 Terraform `user_data` 가 자동
- 수동: STOP/RESET/CHANGE MASTER/START SLAVE

[Phase C] AWS RDS cascade slave 셋업

- RDS 는 SUPER 권한 없으므로 mysql.rds_* procedure 만
- GTID align 신경 써야 (옛 잔재 있을 가능성)

[Phase D] 라이브 검증

- 온프레임 INSERT → DB EC2 + RDS 도달 확인
- /api/system/db-status 가 'primary'/'replica' 응답

2. Phase A: 온프레임 DB (VMware) 준비

A-1. SSH 접속

어떤 방식으로든 온프레임 DB VM 의 root 셸 접속. 방법:

- VMware console 직접
- 또는 ssh ansible@192.168.20.20 (HAProxy subnet route 또는 직접 접근)
- root 비번이 박혀있으면 `su -` 또는 `sudo -i`

```
# [SESSION: 온프레임 DB SSH (root@DB) | at /root]
hostname      # 'DB' 또는 'DB.example.com' 이어야 함
```

A-2. mysql 의 my.cnf include 확인 (⚠ 함정)

```
# [SESSION: 온프레임 DB SSH (root@DB) | at /root]
cat /etc/my.cnf
```

`!includedir /etc/my.cnf.d/` 라인이 있는지 확인. 없으면 추가:

```
# [SESSION: 온프레임 DB SSH (root@DB) | at /root]
echo '' >> /etc/my.cnf
echo '!includedir /etc/my.cnf.d/' >> /etc/my.cnf
```

⚠ 반드시 작은 따옴표 (') 사용: bash 의 ! 가 history expansion 트리거라서 큰 따옴표는 `event not found` 에러. (자세한 트러블슈팅 → 섹션 6.4)

A-3. GTID 활성화

```
# [SESSION: 온프레임 DB SSH (root@DB) | at /root]
cat > /etc/my.cnf.d/gtid.cnf << 'EOF'
[mysqld]
gtid_mode=ON
enforce_gtid_consistency=ON
EOF
```

```
systemctl restart mysqld
sleep 3

# 검증
mysql -u root -p'Qwer1234!' -e "SHOW VARIABLES WHERE Variable_name IN
('gtid_mode', 'enforce_gtid_consistency');"

```

기대 결과:

```
| enforce_gtid_consistency | ON |
| gtid_mode                | ON |

```

💡 **WHY 두 변수 같이:** `gtid_mode=ON` 은 `enforce_gtid_consistency=ON` 없이 시작 거부. 세트로 박아야.

⚠️ **OFF 그대로 떠있으면:** A-2 의 `!includedir` 가 안 들어간 거. `cat /etc/my.cnf` 다시 확인. (자세한 트러블슈팅 → 섹션 6.3)

A-4. inventory DB + app_user + repl_user 생성 + schema 적용

```
# [SESSION: 온프레임 DB SSH (root@DB) | at /root]
mysql -u root -p'Qwer1234!' <<'EOF'
-- DB 생성
CREATE DATABASE IF NOT EXISTS inventory
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

-- App 유저 (Spring Boot 가 접속) + REPLICATION CLIENT 권한도 줘야 (db-status endpoint 용)
CREATE USER IF NOT EXISTS 'app_user'@'%' IDENTIFIED WITH mysql_native_password BY 'Soldesk1.';
GRANT ALL PRIVILEGES ON inventory.* TO 'app_user'@'%';
GRANT REPLICATION CLIENT ON *.* TO 'app_user'@'%';

-- Replication 유저 (DB EC2 가 cascade 받을 때 사용) – SNAT 우회 host '%'
CREATE USER IF NOT EXISTS 'repl_user'@'%' IDENTIFIED WITH mysql_native_password BY 'Qwer1234!';
GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'%';

FLUSH PRIVILEGES;

-- 테이블 생성 (App 레포 sql/schema.sql 그대로)
USE inventory;

CREATE TABLE IF NOT EXISTS product (
    id          BIGINT          NOT NULL AUTO_INCREMENT,
    product_code VARCHAR(50)    NOT NULL,
    name        VARCHAR(100)    NOT NULL,

```

```

        category      VARCHAR(50),
        option_name    VARCHAR(100),
        safety_stock   INT           DEFAULT 0,
        active         TINYINT(1)    DEFAULT 1,
        PRIMARY KEY (id)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS inventory (
    product_id        BIGINT          NOT NULL,
    product_code       VARCHAR(50),
    product_name       VARCHAR(100),
    category           VARCHAR(50),
    current_stock      INT             DEFAULT 0,
    available_stock    INT             DEFAULT 0,
    inbound_stock      INT             DEFAULT 0,
    safety_stock       INT             DEFAULT 0,
    status             VARCHAR(10),
    PRIMARY KEY (product_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS inventory_history (
    id                BIGINT          NOT NULL AUTO_INCREMENT,
    product_id        BIGINT,
    product_code       VARCHAR(50),
    product_name       VARCHAR(100),
    change_type        VARCHAR(20),
    quantity          INT,
    before_stock       INT,
    after_stock        INT,
    reason             VARCHAR(255),
    manager_name       VARCHAR(100),
    created_at         DATETIME,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 검증
SELECT user, host, plugin FROM mysql.user WHERE user IN ('app_user', 'repl_user')
ORDER BY user;
SHOW DATABASES;
SHOW TABLES IN inventory;
EOF

# Master 상태 (cascade 시작점)
mysql -u root -p'Qwer1234!' -e "SHOW MASTER STATUS\G"
```

기대 결과:

- mysql.user 에 app_user@%, repl_user@% 둘 다 mysql_native_password
- inventory DB 존재 + 테이블 3개 (product / inventory / inventory_history)
- Executed_Gtid_Set: <UUID>:1-N (N = 5 정도, schema 생성 트랜잭션)

💡 **WHY `mysql_native_password` 명시:** mysql 8.0 default `caching_sha2_password` 는 SSL/RSA key 교환 필요. cascade replication + Spring Boot HikariCP 둘 다 호환성 위해 native 사용. (섹션 6.5 참고)

💡 **WHY `host '%'` 사용:** Tailscale subnet router 의 SNAT 동작으로 source IP 가 변환됨. mysql `user@host` 매칭 실패 방지. (섹션 6.1 참고)

💡 **WHY `app_user` 에 `REPLICATION CLIENT` 도?:** App 의 `/api/system/db-status` endpoint 가 `SHOW REPLICA STATUS` 실행. 이 명령은 global level 권한 필요. (섹션 6.9 참고)

3. Phase B: AWS DB EC2 cascade slave 셋업

B-1. SSH 접속

```
# [SESSION: db-ec2 SSH | at 아무 디렉토리]
# Bastion 통해 ProxyJump 또는 직접 SSH
hostname # 'ip-10-0-61-10.ap-northeast-2.compute.internal' 같은 EC2 호스트여야
```

⚠️ **반드시 `hostname` 확인:** 잘못된 SSH 세션 (예: 온프레임 DB) 에서 RESET MASTER 등 실행하면 큰 사고. (섹션 6.7 참고)

B-2. mysql 셋업은 `user_data` 가 자동

DB EC2 의 `user_data` (modules/compute/main.tf:280~) 가 자동 셋업한 것:

- mysql 8.0 설치 + root 비번 설정
- `/etc/my.cnf.d/replication.cnf:`

```
[mysqld]
server-id=2
log-bin=mysql-bin
binlog-format=ROW
gtid-mode=ON
enforce-gtid-consistency=ON
log_slave_updates=ON      ← cascade 의 핵심
```

- `repl_user` 자체 (RDS 가 DB EC2 한테 접속할 때 사용) 생성

💡 **`log_slave_updates=ON` 의 의미:** DB EC2 가 master(온프레임)로부터 받은 트랜잭션을 자기 binlog 에도 기록. 그래야 RDS 가 DB EC2 의 binlog 를 읽어서 cascade 가능.

B-3. `server_id` 충돌 확인 (⚠️ 함정)

```
# [SESSION: db-ec2 SSH | at 아무 디렉토리]
DB_PASSWORD='Qwer1234!'

mysql -u root -p"$DB_PASSWORD" -e "STOP SLAVE;" 2>/dev/null
```

```

SERVER_ID=$(mysql -u root -p"$DB_PASSWORD" -BNe "SELECT @@server_id;")
echo "현재 server_id: $SERVER_ID"

# 온프레임 (server_id=1) 과 같으면 변경
if [ "$SERVER_ID" = "1" ]; then
    echo "⚠ 온프레임과 충돌 - 2 로 변경"
    mysql -u root -p"$DB_PASSWORD" -e "SET PERSIST server_id=2;"
fi

```

💡 **WHY:** master/slave 의 server_id 같으면 mysql 이 self-loop 인식해서 IO thread 거부. 온프레임 = 1, DB EC2 = 2 가 표준.

B-4. Cascade slave 셋업

```

# [SESSION: db-ec2 SSH | at 아무 디렉토리]
DB_PASSWORD='Qwer1234!'

mysql -u root -p"$DB_PASSWORD" <<'EOF'
RESET SLAVE ALL;
RESET MASTER;          -- DB EC2 의 옛 GTID 셋 비움 (있으면 충돌)

CHANGE MASTER TO
    MASTER_HOST='192.168.20.20',
    MASTER_USER='repl_user',
    MASTER_PASSWORD='Qwer1234!',
    MASTER_AUTO_POSITION=1;

START SLAVE;
EOF

sleep 5

# 검증
mysql -u root -p"$DB_PASSWORD" -e "SHOW SLAVE STATUS\G" | grep -E
"Master_Host|Slave_IO_Running|Slave_SQL_Running|Seconds_Behind_Master|Last_IO_Error|Retrieved_Gtid_Set|Executed_Gtid_Set"

mysql -u root -p"$DB_PASSWORD" -e "SHOW MASTER STATUS\G"

# inventory DB 도달했는지
mysql -u root -p"$DB_PASSWORD" -e "USE inventory; SHOW TABLES;"

```

기대 결과:

```

Slave_IO_Running: Yes          ← 핵심
Slave_SQL_Running: Yes        ← 핵심
Seconds_Behind_Master: 0

```

```
Retrieved_Gtid_Set: <온프레임 UUID>:1-N
Executed_Gtid_Set: <온프레임 UUID>:1-N
```

inventory DB + 테이블 3개 도달.

⚠ 에러 케이스:

- `Access denied for user 'repl_user'@'_gateway'` → 온프레임에 `repl_user@'%'` 없음 (섹션 6.1)
- `Got fatal error 1236: could not find next log` → master 의 binlog 변경 후 잘못된 위치 (섹션 6.7)
- `Replica has more GTIDs than the source` → `server_id` 충돌 또는 GTID 잔재 (섹션 6.6, 6.8)

4. Phase C: AWS RDS cascade slave 셋업

C-1. SSH 또는 mysql client 준비

Jenkins EC2 에서 진행 (RDS 에 접근 가능한 보안 그룹 허용된 머신):

```
# [SESSION: Jenkins EC2 SSH | at 아무 디렉토리]
hostname      # 'ip-10-0-41-31...' 같은 Jenkins 호스트

# 변수 셋업
RDS_ENDPOINT=$(cd /var/lib/jenkins/workspace/hybrid-dr-pipeline/project-springboot-dev/root/dr && sudo -u jenkins terraform output -raw rds_endpoint)
DB_EC2_IP=$(cd /var/lib/jenkins/workspace/hybrid-dr-pipeline/project-springboot-dev/root/dr && sudo -u jenkins terraform output -raw db_ec2_private_ip)
DB_PASSWORD=$(sudo grep '^db_password' /etc/hybrid-dr/terraform.tfvars | cut -d '=' -f2- | tr -d '\r\n')

echo "RDS_ENDPOINT: $RDS_ENDPOINT"
echo "DB_EC2_IP: $DB_EC2_IP"
```

C-2. RDS replication 셋업

```
# [SESSION: Jenkins EC2 SSH | at 아무 디렉토리]
mysql -h "$RDS_ENDPOINT" -u admin -p"$DB_PASSWORD" <<EOF
CALL mysql.rds_stop_replication;
CALL mysql.rds_reset_external_master;
CALL mysql.rds_set_external_master_with_auto_position(
    '$DB_EC2_IP', 3306, 'repl_user', 'Qwer1234!', 0, 0
);
CALL mysql.rds_start_replication;
EOF

sleep 15

# 검증
```

```
mysql -h "$RDS_ENDPOINT" -u admin -p"$DB_PASSWORD" -e "SHOW SLAVE STATUS\G" | grep
-E
"Master_Host|Slave_IO_Running|Slave_SQL_Running|Seconds_Behind_Master|Last_IO_Erro
r|Retrieved_Gtid_Set|Executed_Gtid_Set"
```

💡 *WHY mysql.rds_procedure**: AWS RDS 는 SUPER 권한 차단. 일반 mysql 의 `STOP SLAVE; CHANGE MASTER TO ...` 거부. AWS 가 미리 막아둔 procedure 만 사용 가능.

💡 **인자 순서**: (host, port, user, password, ssl_encryption, delay). 마지막 두 개 0, 0 = SSL disable + delay 0초.

기대 결과:

```
Slave_IO_Running: Yes
Slave_SQL_Running: Yes
Retrieved_Gtid_Set: <DB EC2 발행 GTID 셋>
```

⚠ 에러 케이스 (가장 흔함):

- *Replica has more GTIDs than the source* (ERROR 1644) → RDS 가 옛 GTID 잔재 가지고 있음. **섹션 6.8 (Multi-node GTID align trick) 필수.**

5. Phase D: 라이브 검증

D-1. 온프레임에서 테스트 INSERT

```
# [SESSION: 온프레임 DB SSH (root@DB) | at /root]
mysql -u root -p'Qwer1234!' <<'EOF'
USE inventory;
INSERT INTO product (product_code, name, category, safety_stock, active)
VALUES ('CASCADE-TEST', '검증용 제품', '테스트', 10, 1);
EOF

mysql -u root -p'Qwer1234!' -e "SHOW MASTER STATUS\G" | grep Executed_Gtid_Set
```

D-2. 5초 후 RDS 에서 도달 확인

```
# [SESSION: Jenkins EC2 SSH | at 아무 디렉토리]
sleep 5
mysql -h "$RDS_ENDPOINT" -u admin -p"$DB_PASSWORD" -e "SELECT * FROM
inventory.product WHERE product_code='CASCADE-TEST';"
```

도달하면 cascade 동작. 안 도달하면 **섹션 6.6 / 6.8** 진단.

D-3. App endpoint 검증

```
# [SESSION: Windows PowerShell | at 아무 디렉토리]
curl.exe -s http://solodeskcloud.xyz/api/system/db-status
```

기대:

```
{"connectedHost":"192.168.20.20:3306","replicationLag":null,"replicationStatus":"primary"}
```

⚠ **replicationStatus: "unknown"** 뜨면: app_user 의 REPLICATION CLIENT 권한 부족 (섹션 6.9).

6. 트러블슈팅 가이드

6.1. Access denied for user 'repl_user'@'_gateway'

증상: cascade slave (DB EC2) 의 SHOW SLAVE STATUS 에서:

```
Last_IO_Error: Access denied for user 'repl_user'@'_gateway' (using password: YES)
```

원인: HAProxy 가 Tailscale subnet router 로 동작 시 default SNAT 으로 source IP 변환. 온프레미 DB 가 source IP 의 reverse DNS 풀려다 실패 → placeholder `_gateway` 로 매칭. mysql.user 의 `repl_user@<특정IP>` 와 안 맞음.

진단:

```
SELECT user, host FROM mysql.user WHERE user='repl_user';
-- 만약 host 가 '%' 가 아니라 특정 IP 면 문제
```

해결:

```
DROP USER IF EXISTS 'repl_user'@'<특정IP>';
CREATE USER 'repl_user'@'%' IDENTIFIED WITH mysql_native_password BY 'Qwer1234!';
GRANT REPLICATION SLAVE ON *.* TO 'repl_user'@'%';
FLUSH PRIVILEGES;
```

6.2. gtid_mode: OFF (인프라 코드는 GTID 전제인데)

증상: master 의 mysql 변수가 `gtid_mode: OFF`. cascade slave 의 `MASTER_AUTO_POSITION=1` 안 동작.

원인: mysql 8.0 default 가 GTID OFF. 명시적으로 활성화 안 하면 OFF.

해결: 섹션 A-3 참고. `gtid_mode=ON + enforce_gtid_consistency=ON` 둘 다 박아야.

6.3. mysql.cnf 의 `!includedir` 미설정

증상: `/etc/my.cnf.d/*.cnf` 에 박은 설정이 무시됨. mysql 재시작 후 변수 그대로.

진단:

```
cat /etc/my.cnf | grep '!includedir'
# 결과 없으면 문제
```

원인: RPM 으로 mysql 설치할 때 `/etc/my.cnf` 끝에 자동 박혀야 할 `!includedir /etc/my.cnf.d/` 가 안 박힌 환경 존재.

해결:

```
echo '' >> /etc/my.cnf
echo '!includedir /etc/my.cnf.d/' >> /etc/my.cnf
systemctl restart mysqld
```

6.4. bash `!includedir: event not found`

증상:

```
echo "!includedir /etc/my.cnf.d/" >> /etc/my.cnf
# bash: !includedir: event not found
```

원인: bash 의 `!` 가 history expansion 트리거. 큰 따옴표 ("`...`") 안에서도 활성화. 작은 따옴표만 안전.

해결: 작은 따옴표 사용:

```
echo '!includedir /etc/my.cnf.d/' >> /etc/my.cnf
```

6.5. mysql 8.0 의 `caching_sha2_password` 호환성

증상: replication 또는 Spring Boot connection 시 알 수 없는 인증 실패.

원인: mysql 8.0 default 인증 plugin = `caching_sha2_password` (SSL/RSA key 교환 필요). 일부 클라이언트/replication 환경에서 호환성 문제.

해결: 모든 user 생성 시 `mysql_native_password` 명시:

```
CREATE USER 'app_user'@'%' IDENTIFIED WITH mysql_native_password BY 'Soldesk1.';
```

또는 기존 user 변경:

```
ALTER USER 'app_user'@'%' IDENTIFIED WITH mysql_native_password BY 'Soldesk1.';  
FLUSH PRIVILEGES;
```

6.6. server_id 충돌 (source and replica have equal MySQL server ids)

증상:

```
Last_IO_Error: Fatal error: The replica I/O thread stops because source and  
replica have equal MySQL server ids
```

원인: master 와 slave 의 server_id 가 같음. 또는 자기 자신을 master 로 가리키는 잘못된 설정.

진단:

```
SHOW SLAVE STATUS\G      -- Master_Host 가 자기 IP 면 잘못된 설정  
SELECT @@server_id;      -- master 와 같으면 충돌
```

해결 1 (자기 자신을 slave 로 만든 경우):

```
STOP SLAVE;  
RESET SLAVE ALL;  
-- 그 후 정상 cascade 셋업 다시 (섹션 B-4 또는 C-2)
```

해결 2 (server_id 같은 경우):

```
STOP SLAVE;  
SET PERSIST server_id=2;  
-- 그 후 cascade 다시 시작
```

6.7. Got fatal error 1236: could not find next log

증상:

```
Got fatal error 1236 from source when reading data from binary log: 'could not find next log; the first event '' at 4, the last event read from './binlog.000018' at 6727'
```

원인: master 의 binlog 가 reset 되거나 purge 됨 (예: 누군가 RESET MASTER 실행). slave 가 옛 binlog 위치를 기억하다 못 찾음.

해결: slave 에서 RESET 후 처음부터 다시:

```
STOP SLAVE;
RESET SLAVE ALL;
RESET MASTER;
CHANGE MASTER TO MASTER_HOST=..., MASTER_AUTO_POSITION=1;
START SLAVE;
```

⚠️ 예방: master 의 RESET MASTER 는 매우 신중히. 잘못된 SSH 세션에서 실행하면 큰 사고 (자기 자신 binlog 다 날림). 매 명령 전 **hostname** 확인 권장.

6.8. ❤️ RDS 의 GTID 잔재 + Multi-node align trick (가장 어려운 케이스)

증상:

- RDS 에서 `CALL mysql.rds_set_external_master_with_auto_position(...)` 호출 후
- `Slave_IO_Running: Yes, Slave_SQL_Running: Yes` 정상
- 하지만 `Retrieved_Gtid_Set: (빈 값)`, 새 INSERT 도 RDS 에 안 도달

진단 1단계 (master = DB EC2 측):

```
SHOW PROCESSLIST;
-- 'Binlog Dump GTID' thread 의 status:
-- "Source has sent all binlog to replica; waiting for more updates" ← master 는 다 보냈다고 보고
```

진단 2단계 (RDS 측):

```
SHOW SLAVE STATUS\G
-- Executed_Gtid_Set 자세히 봄. master 셋과 비교.
```

원인 (가능성 가장 높음):

- RDS 의 `Executed_Gtid_Set` 에 옛 cascade 시도들에서 누적된 가짜 GTID (예: `<UUID>:1-18`)
- master 의 셋 (`<UUID>:1-N`, N 이 작음) 이 RDS 셋의 **부분집합**
- mysql GTID auto-position: 보낼 `GTID = master 셋 - slave 셋`

- 결과 = 빈 셋 → master 가 "보낼 거 없음" 판단
- 새 INSERT 도 RDS 가 "이미 가지고 있다" 며 skip

해결: Multi-node GTID align:

1. master (DB EC2) 의 GTID 셋을 RDS 셋과 같은 수준으로 늘림:

```
-- DB EC2 에서. RDS 가 가진 것까지 (예: 1-18) empty trans 박기.
-- master 가 이미 가진 GTID (예: 1-5) 는 건너뛰고 6, 7, ..., 18 까지.
SET GTID_NEXT='<UUID>:6'; BEGIN; COMMIT;
SET GTID_NEXT='<UUID>:7'; BEGIN; COMMIT;
... (RDS 셋 끝까지)
SET GTID_NEXT='AUTOMATIC';
```

2. 온프레임 master 도 동일하게 align:

- 온프레임이 발행한 GTID (예: 1-5 의 schema + 6 의 INSERT 같은 거) 는 그대로
- RDS 가 가진 미사용 GTID (예: 7-18) 만 empty trans
- 그러면 온프레임 셋 = DB EC2 셋 = RDS 셋

3. 새 INSERT → 다음 GTID (예: 19) 발행 → master/slave 셋이 RDS 보다 커짐 → master 가 RDS 한테 19 보냄 → cascade 흐름 시작.

검증:

```
-- RDS 에서 새 INSERT 결과 확인
SELECT * FROM inventory.product WHERE product_code = '<새 INSERT 의 코드>';

-- RDS Retrieved_Gtid_Set 변화:
SHOW SLAVE STATUS\G -- Retrieved_Gtid_Set 에 새 GTID 있어야
```

💡 **부작용 인지:** align trick 으로 박은 empty GTID 의 "원래 데이터" (예: GTID 6 의 진짜 INSERT) 는 RDS 에 도달 못 함. 발표/시연용으로는 align 후의 새 INSERT 부터 보여주는 게 자연스러움.

💡 **방지 (처음 셋업 시):** RDS 가 처음 가동될 때 cascade 시도를 여러 번 하지 말 것. 한 번에 정확하게.

6.9. replicationStatus: "unknown" (App /api/system/db-status)

증상: App 의 /api/system/db-status 가 {"replicationStatus":"unknown"} 응답.

원인: App 코드 (SystemStatusController.java) 의 catch 블록에서 발생. SHOW REPLICA STATUS 실행 시 exception. 즉 app_user 의 권한 부족.

해결 1단계: app_user 에 REPLICATION CLIENT GRANT

```
GRANT REPLICATION CLIENT ON *.* TO 'app_user'@'%';
```

```
FLUSH PRIVILEGES;
```

해결 2단계 (즉시 반영 안 되면): HikariCP cached connection 때문. 30초 기다림 또는 Spring Boot 재시작.

Spring Boot 재시작 (Jenkins SSH 에서 ansible 사용):

```
cd /var/lib/jenkins/workspace/hybrid-dr-pipeline/Ansible
sudo -u jenkins ansible -i inventories/on-premise-phase1/hosts.yml webwas \
  -m systemd -a "name=logistics state=restarted" --become
```

 **WHY 30초 기다림:** HikariCP 가 idle connection 정리 후 새 connection 만드는 데 시간 걸림. mysql 권한 변경은 새 connection 부터 반영. 기존 connection 은 cached 권한 그대로.

7. 부록 A: 발표 라이브 시연 명령

시연 시나리오: "온프레임 INSERT → RDS 까지 5초 안에 cascade"

1) 시작 전 — 현재 상태 확인 (PowerShell):

```
# [SESSION: Windows PowerShell | at 아무 디렉토리]
curl.exe -s http://solodeskcloud.xyz/api/system/db-status
# 기대: {"connectedHost":"192.168.20.20:3306","replicationStatus":"primary"}
```

2) 온프레임에 INSERT (root@DB):

```
# [SESSION: 온프레임 DB SSH (root@DB) | at /root]
mysql -u root -p'Qwer1234!' -e "
USE inventory;
INSERT INTO product (product_code, name, category, safety_stock, active)
VALUES ('DEMO-001', '발표 시연 제품', '데모', 10, 1);
SELECT * FROM product WHERE product_code='DEMO-001';
"
```

3) 5초 대기 후 RDS 에서 도달 확인 (Jenkins SSH):

```
# [SESSION: Jenkins EC2 SSH | at 아무 디렉토리]
sleep 5
RDS_ENDPOINT=$(cd /var/lib/jenkins/workspace/hybrid-dr-pipeline/project-
springboot-dev/root/dr && sudo -u jenkins terraform output -raw rds_endpoint)
DB_PASSWORD=$(sudo grep '^db_password' /etc/hybrid-dr/terraform.tfvars | cut -d'='
-f2- | tr -d ' "\r\n')

mysql -h "$RDS_ENDPOINT" -u admin -p"$DB_PASSWORD" -e "
```

```
SELECT * FROM inventory.product WHERE product_code='DEMO-001';
"
```

4) 발표 narrative:

- "지금 온프레임 DB 에 새 제품 등록"
- "5초 후 AWS RDS 에서 같은 데이터 확인"
- "이게 cascade replication 의 라이브 동작 — 데이터 손실 없음"

8. 부록 B: SSH 세션 / 머신 / IP 매트릭스

세션	머신	hostname (검증용)	IP	mysql role
온프레임 DB SSH	VMware DB VM	DB 또는 DB.example.com	192.168.20.20	master
온프레임 WEBWAS SSH	VMware WEBWAS VM	(다양)	192.168.20.12	(Spring Boot 호스트)
온프레임 HAProxy SSH	VMware HAProxy VM	(다양)	192.168.10.3 / 20.2	Tailscale subnet router
Bastion SSH	AWS Bastion EC2	ip-10-0-1-*	10.0.1.x (public IP 변동)	(jump host)
Jenkins EC2 SSH	AWS Jenkins EC2	ip-10-0-41-31	10.0.41.31	mysql client (RDS 접근)
db-ec2 SSH	AWS DB EC2	ip-10-0-61-10	10.0.61.10	intermediate slave + master
(RDS 직접 SSH 불가)	AWS RDS	(managed)	RDS endpoint	final slave

자주 쓰는 변수

변수	값	위치
온프레임 DB IP	192.168.20.20	hardcoded
DB EC2 IP	10.0.61.10 (terraform output)	tfvars
RDS endpoint	<...>.rds.amazonaws.com (terraform output)	tfvars
repl_user 비번	Qwer1234!	tfvars db_password
app_user 비번	Soldesk1.	App 레포 application.yml + 인프라 코드 hardcode
mysql root 비번 (온프레임)	Qwer1234!	Ansible vault

9. 부록 C: 진단 명령 cheat-sheet

Master 측 진단 (cascade 가 막힐 때)

```
SHOW VARIABLES LIKE 'gtid_mode';
SHOW VARIABLES LIKE 'gtid_purged';
SHOW MASTER STATUS\G
SHOW BINLOG EVENTS IN '<binlog 파일>' LIMIT 30;
SHOW PROCESSLIST;           -- slave 의 'Binlog Dump GTID' thread 보여야
```

Slave 측 진단

```
SHOW SLAVE STATUS\G
-- 핵심 필드:
-- Slave_IO_Running, Slave_SQL_Running (둘 다 Yes 여야)
-- Last_IO_Error, Last_SQL_Error (빈 값이어야)
-- Master_Host, Master_Log_File, Read_Master_Log_Pos
-- Retrieved_Gtid_Set (실제로 받은 GTID)
-- Executed_Gtid_Set (적용된 GTID)
-- Seconds_Behind_Master (lag)
```

RDS 전용

```
CALL mysql.rds_show_external_master;    -- 만약 있다면, master 설정 보기
SHOW GRANTS;                           -- admin 의 권한 확인 (SUPER 없음 확인)
```

이 매뉴얼 위치

C:\Users\cpj32\Desktop\프로젝트\Git\dr-project-cascade-replication-manual.md

처음부터 셋업할 때 섹션 0 → 2 → 3 → 4 → 5 순서로. 막히면 섹션 6 의 트러블슈팅 가이드.

질문이 생기면 (잘 동작하던 게 갑자기 안 될 때 등) [dr-project-review-2026-04-26-day-summary.md](#) 의 사례 narrative 도 참고 — 우리가 실제로 부딪힌 시나리오.